🔥 Top 20 Real-Industry Scenarios for Power BI Automation with GitHub Copilot

author: Sudhanshu Mukherjee
Date: 09/17/2025

A. Data Engineering & Preparation

1. Automated ETL Script Generation

- Copilot generates Python or SQL scripts to clean, join, and transform raw data (CSV, APIs, Databases) before loading into Power BI.
- Example: Automatically creating a SQL stored procedure for aggregating sales by region.

Prompt:

```
Generate comprehensive ETL scripts for Power BI data pipeline using global
sales data with these requirements:

1. Python ETL Scripts:
   - Read global_sales.xlsx with Orders, Returns, People sheets
   - Clean and standardize data (remove duplicates, handle nulls, format
dates)
   - Join tables using proper relationships (customer ID, product ID, order
ID)
   - Create calculated columns: profit margin, sales categories, customer
tiers
   - Export cleaned data as single CSV file ready for Power BI

2. SQL Stored Procedures:
   - Aggregate sales data by region, category, time periods
    - Create summary tables optimized for Power BI dashboards
   - Add data quality checks and validation rules

   Output ready-to-use Python script, SQL procedure, etc for immediate
implementation
```

Python Code:

```python
# Python ETL Script for Power BI Data Pipeline
# filepath: d:\Neokai\Github_PBI\AgenticPowerBI\etl_global_sales.py

import pandas as pd
```

```
import numpy as np
from datetime import datetime

def clean_global_sales():
    try:
        print("Starting ETL process...")

        # Read Excel sheets
        print("Reading Excel sheets...")
        orders = pd.read_excel('global_sales.xlsx', sheet_name='Orders')
        returns = pd.read_excel('global_sales.xlsx', sheet_name='Returns')
        people = pd.read_excel('global_sales.xlsx', sheet_name='People')

        print(f"Orders shape: {orders.shape}")
        print(f"Returns shape: {returns.shape}")
        print(f"People shape: {people.shape}")

        # Clean and standardize Orders
        print("Cleaning Orders data...")

        # Remove duplicates
        initial_rows = len(orders)
        orders = orders.drop_duplicates()
        print(f"Removed {initial_rows - len(orders)} duplicate rows")

        # Handle missing values in critical columns
        orders = orders.dropna(subset=['Order ID', 'Customer ID', 'Product
ID', 'Sales', 'Profit'])
        print(f"Final Orders rows after cleanup: {len(orders)}")



        # Process date columns (already in datetime format)
        print("Processing date columns...")
        orders['Order Date'] = pd.to_datetime(orders['Order Date'],
errors='coerce')
        orders['Ship Date'] = pd.to_datetime(orders['Ship Date'],
errors='coerce')

        # Create calculated columns
        print("Creating calculated columns...")

        # Profit Margin
        orders['Profit Margin'] = np.where(orders['Sales'] != 0,
orders['Profit'] / orders['Sales'], 0)

        # Sales Category based on order value
        orders['Sales Category'] = pd.cut(orders['Sales'],
                                          bins=[0, 100, 500, 1500, np.inf],
                                          labels=['Low', 'Medium', 'High',
'Very High'])
```

```
        # Customer Tier based on total customer sales
        print("Creating customer tiers...")
        customer_sales = orders.groupby('Customer ID')
['Sales'].sum().reset_index()
        customer_sales.rename(columns={'Sales': 'Customer_Total_Sales'},
inplace=True)

        # Create quartile-based tiers
        customer_sales['Customer Tier'] =
pd.qcut(customer_sales['Customer_Total_Sales'],
                                                4,
                                                labels=['Bronze', 'Silver',
'Gold', 'Platinum'],
                                                duplicates='drop')

        # Merge customer tier back to orders
        orders = orders.merge(customer_sales[['Customer ID', 'Customer
Tier', 'Customer_Total_Sales']],
                              on='Customer ID',
                              how='left')

        # Join Returns data
        print("Joining returns data...")
        orders = orders.merge(returns[['Order ID', 'Returned']],
                              on='Order ID',
                              how='left')

        # Since there's no direct salesperson mapping, we'll join People by
Region
        print("Joining people data by region...")
        # This creates a region-to-person mapping (approximate)
        orders = orders.merge(people[['Person', 'Region']],
                              on='Region',
                              how='left',
                              suffixes=('', '_Manager'))

        # Fill missing values
        print("Filling missing values...")
        orders['Returned'] = orders['Returned'].fillna('No')
        orders['Customer Tier'] = orders['Customer Tier'].fillna('Bronze')

        # Create additional useful columns
        orders['Year'] = orders['Order Date'].dt.year
        orders['Month'] = orders['Order Date'].dt.month
        orders['Quarter'] = orders['Order Date'].dt.quarter
        orders['Weekday'] = orders['Order Date'].dt.day_name()

        # Calculate delivery time
        orders['Delivery Days'] = (orders['Ship Date'] - orders['Order
```

```
Date']).dt.days

        # Data quality summary
        print("\n=== DATA QUALITY SUMMARY ===")
        print(f"Total rows: {len(orders)}")
        print(f"Date range: {orders['Order Date'].min()} to {orders['Order
Date'].max()}")
        print(f"Unique customers: {orders['Customer ID'].nunique()}")
        print(f"Unique products: {orders['Product ID'].nunique()}")
        print(f"Total sales: ${orders['Sales'].sum():,.2f}")
        print(f"Average profit margin: {orders['Profit
Margin'].mean():.1%}")
        print(f"Orders with returns: {(orders['Returned'] == 'Yes').sum()}")

        # Export cleaned data
        print("\nExporting cleaned data...")
        output_file = 'global_sales_cleaned.csv'
        orders.to_csv(output_file, index=False)
        print(f"✅ Cleaned data exported to: {output_file}")

        # Create separate summary tables for Power BI
        print("Creating summary tables...")

        # Regional summary
        regional_summary = orders.groupby(['Region', 'Year',
'Quarter']).agg({
            'Sales': 'sum',
            'Profit': 'sum',
            'Order ID': 'nunique',
            'Customer ID': 'nunique'
        }).reset_index()
        regional_summary.columns = ['Region', 'Year', 'Quarter',
'Total_Sales', 'Total_Profit', 'Order_Count', 'Customer_Count']
        regional_summary.to_csv('regional_summary.csv', index=False)

        # Customer summary
        customer_summary = orders.groupby('Customer ID').agg({
            'Sales': 'sum',
            'Profit': 'sum',
            'Order ID': 'nunique',
            'Customer Tier': 'first',
            'Customer Name': 'first'
        }).reset_index()
        customer_summary.columns = ['Customer_ID', 'Total_Sales',
'Total_Profit', 'Order_Count', 'Customer_Tier', 'Customer_Name']
        customer_summary.to_csv('customer_summary.csv', index=False)

        print("✅ Summary tables created: regional_summary.csv,
customer_summary.csv")
        print("✅ ETL process completed successfully!")
```

```
        return orders

    except Exception as e:
        print(f"❌ Error in ETL process: {str(e)}")
        return None

# Run the ETL process
if __name__ == "__main__":
    cleaned_data = clean_global_sales()
```

SQL Code:

```
-- SQL Stored Procedure for Power BI Aggregation
-- filepath: AgenticPowerBI\sp_aggregate_global_sales_.sql

CREATE OR ALTER PROCEDURE sp_AggregateGlobalSales
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        PRINT 'Starting Global Sales Aggregation...'

        -- Data Quality Checks
        PRINT 'Performing data quality checks...'
        IF EXISTS (SELECT 1 FROM Orders WHERE Sales IS NULL OR Profit IS
NULL)
        BEGIN
            RAISERROR('Null values found in Sales or Profit columns.', 16,
1)
            RETURN
        END

        -- Drop existing summary tables if they exist
        IF OBJECT_ID('SalesSummaryByRegionCategoryMonth', 'U') IS NOT NULL
            DROP TABLE SalesSummaryByRegionCategoryMonth

        IF OBJECT_ID('SalesSummaryByCustomerSegment', 'U') IS NOT NULL
            DROP TABLE SalesSummaryByCustomerSegment

        IF OBJECT_ID('SalesSummaryByProduct', 'U') IS NOT NULL
            DROP TABLE SalesSummaryByProduct

        -- Aggregate by Region, Category, Month
        PRINT 'Creating regional/category/monthly summary...'
        SELECT
            o.Region,
```

```
            o.Category,
            YEAR(o.[Order Date]) AS SalesYear,
            MONTH(o.[Order Date]) AS SalesMonth,
            DATENAME(MONTH, o.[Order Date]) AS MonthName,
            COUNT(*) AS TransactionCount,
            COUNT(DISTINCT o.[Order ID]) AS TotalOrders,
            COUNT(DISTINCT o.[Customer ID]) AS UniqueCustomers,
            SUM(o.Sales) AS TotalSales,
            SUM(o.Profit) AS TotalProfit,
            SUM(o.Quantity) AS TotalQuantity,
            AVG(o.Sales) AS AvgOrderValue,
            AVG(o.Profit / NULLIF(o.Sales, 0)) AS AvgProfitMargin,
            MIN(o.[Order Date]) AS FirstOrderDate,
            MAX(o.[Order Date]) AS LastOrderDate
        INTO SalesSummaryByRegionCategoryMonth
        FROM Orders o
        WHERE o.[Order Date] IS NOT NULL
        GROUP BY o.Region, o.Category, YEAR(o.[Order Date]), MONTH(o.[Order
Date]), DATENAME(MONTH, o.[Order Date])

        -- Aggregate by Customer Segment (using Segment column instead of
non-existent Customer Tier)
        PRINT 'Creating customer segment summary...'
        SELECT
            o.Segment AS CustomerSegment,
            o.Region,
            COUNT(*) AS TransactionCount,
            COUNT(DISTINCT o.[Order ID]) AS TotalOrders,
            COUNT(DISTINCT o.[Customer ID]) AS UniqueCustomers,
            SUM(o.Sales) AS TotalSales,
            SUM(o.Profit) AS TotalProfit,
            SUM(o.Quantity) AS TotalQuantity,
            AVG(o.Sales) AS AvgOrderValue,
            AVG(o.Profit / NULLIF(o.Sales, 0)) AS AvgProfitMargin
        INTO SalesSummaryByCustomerSegment
        FROM Orders o
        WHERE o.Segment IS NOT NULL
        GROUP BY o.Segment, o.Region

        -- Aggregate by Product Performance
        PRINT 'Creating product performance summary...'
        SELECT
            o.Category,
            o.[Sub-Category],
            o.[Product Name],
            o.Region,
            COUNT(*) AS TransactionCount,
            COUNT(DISTINCT o.[Order ID]) AS TotalOrders,
            COUNT(DISTINCT o.[Customer ID]) AS UniqueCustomers,
            SUM(o.Sales) AS TotalSales,
```

```
            SUM(o.Profit) AS TotalProfit,
            SUM(o.Quantity) AS TotalQuantity,
            AVG(o.Sales) AS AvgOrderValue,
            AVG(o.Profit / NULLIF(o.Sales, 0)) AS AvgProfitMargin,
            -- Performance metrics
            RANK() OVER (PARTITION BY o.Category ORDER BY SUM(o.Sales) DESC)
AS SalesRankInCategory,
            RANK() OVER (PARTITION BY o.Category ORDER BY SUM(o.Profit)
DESC) AS ProfitRankInCategory
        INTO SalesSummaryByProduct
        FROM Orders o
        WHERE o.[Product Name] IS NOT NULL
        GROUP BY o.Category, o.[Sub-Category], o.[Product Name], o.Region

        -- Data validation checks
        PRINT 'Performing validation checks...'

        -- Check for negative sales
        IF EXISTS (SELECT 1 FROM SalesSummaryByRegionCategoryMonth WHERE
TotalSales < 0)
        BEGIN
            RAISERROR('Negative sales found in regional summary table.', 16,
1)
            RETURN
        END

        -- Check for unrealistic profit margins
        IF EXISTS (SELECT 1 FROM SalesSummaryByRegionCategoryMonth WHERE
AvgProfitMargin > 1 OR AvgProfitMargin < -1)
        BEGIN
            PRINT 'Warning: Extreme profit margins detected (>100% or
<-100%)'
        END

        -- Summary statistics
        DECLARE @TotalRows INT, @TotalSales MONEY, @TotalProfit MONEY

        SELECT
            @TotalRows = COUNT(*),
            @TotalSales = SUM(TotalSales),
            @TotalProfit = SUM(TotalProfit)
        FROM SalesSummaryByRegionCategoryMonth

        PRINT '=== AGGREGATION SUMMARY ==='
        PRINT 'Regional/Category/Month records: ' + CAST(@TotalRows AS
VARCHAR)
        PRINT 'Total Sales: $' + FORMAT(@TotalSales, 'N2')
        PRINT 'Total Profit: $' + FORMAT(@TotalProfit, 'N2')
        PRINT 'Overall Profit Margin: ' + FORMAT(@TotalProfit/@TotalSales,
'P2')
```

```
        SELECT
            @TotalRows = COUNT(*)
        FROM SalesSummaryByCustomerSegment
        PRINT 'Customer Segment records: ' + CAST(@TotalRows AS VARCHAR)

        SELECT
            @TotalRows = COUNT(*)
        FROM SalesSummaryByProduct
        PRINT 'Product Performance records: ' + CAST(@TotalRows AS VARCHAR)

        PRINT '✅ Global Sales Aggregation completed successfully!'

    END TRY
    BEGIN CATCH
        PRINT '❌ Error in aggregation process:'
        PRINT 'Error Number: ' + CAST(ERROR_NUMBER() AS VARCHAR)
        PRINT 'Error Message: ' + ERROR_MESSAGE()
        THROW
    END CATCH
END
```

How it works

- First import your Excel data to SQL Server
- Then use the fixed SQL script above
- More complexity but works if you need database integration

2. Dynamic Date Table Creation

- Copilot writes DAX or M script to generate a dynamic calendar table (with Fiscal Year, Week, Holidays).
- Saves manual coding time and ensures consistency.

Prompt

```
Create dynamic date table generation scripts for Power BI with these
requirements:

1. DAX Date Table Creation:
   - Generate date range from 2010 to 2030
   - Include standard date hierarchy: Year, Quarter, Month, Week
   - Add fiscal year columns (fiscal year starts April 1st)
   - Create weekday/weekend indicators
   - Add holiday flags for major US holidays

2. M Language Date Table (Power Query):
```

```
   - Dynamic date range based on data min/max dates
   - Business day calculations (exclude weekends)
   - Week numbering (ISO week and US week)
   - Season indicators (Spring, Summer, Fall, Winter)
   - Date formatting variations (Short date, Long date, etc.)

3. Key Date Columns to Include:
   - Date, Year, Quarter, Month, Week
   - Fiscal Year, Fiscal Quarter, Fiscal Month
   - Day of Week, Day Name, Month Name
   - Is Weekend, Is Holiday, Is Business Day
   - Week Start Date, Month Start Date, Quarter Start Date
   - Days from Today, Relative Period indicators

4. Time Intelligence Support:
   - Previous Year, Previous Quarter, Previous Month
   - Year-to-Date, Quarter-to-Date, Month-to-Date indicators
   - Same period last year calculations
   - Rolling period indicators (Last 30 days, Last 90 days)

Output complete DAX table creation code and M language Power Query script
ready to use in Power BI.
```

// DAX Date Table Creation (2010-2030, Fiscal Year starts April 1st, US Holidays, Weekday/Weekend, Time Intelligence)

```
Date-New =
ADDCOLUMNS (
    CALENDAR (DATE(2010,1,1), DATE(2030,12,31)),
    "Date ID", FORMAT([Date], "YYYYMMDD"),
    "Year", YEAR([Date]),
    "Quarter", QUARTER([Date]),
    "Month", MONTH([Date]),
    "Month Name", FORMAT([Date], "MMMM"),
    "Month Short", FORMAT([Date], "MMM"),
    "Week", WEEKNUM([Date], 1),
    "ISO Week", WEEKNUM([Date], 21),
    "Day", DAY([Date]),
    "Day Name", FORMAT([Date], "dddd"),
    "Day Short", FORMAT([Date], "ddd"),
    "Is Weekend", WEEKDAY([Date],2) > 5,

    -- Fiscal Year (April 1st start)
    "Fiscal Year", IF(MONTH([Date]) >= 4, YEAR([Date]) + 1, YEAR([Date])),
    "Fiscal Quarter", SWITCH(TRUE(),
        MONTH([Date]) >= 4 && MONTH([Date]) <= 6, 1,
        MONTH([Date]) >= 7 && MONTH([Date]) <= 9, 2,
        MONTH([Date]) >= 10 && MONTH([Date]) <= 12, 3,
```

```
        4
    ),
    "Fiscal Month", IF(MONTH([Date]) >= 4, MONTH([Date]) - 3, MONTH([Date])
+ 9),
    "Fiscal Year Label", "FY" & IF(MONTH([Date]) >= 4, YEAR([Date]) + 1,
YEAR([Date])),

    -- Date Boundaries
    "Week Start Date", [Date] - WEEKDAY([Date],2) + 1,
    "Month Start Date", DATE(YEAR([Date]), MONTH([Date]), 1),
    "Quarter Start Date", DATE(YEAR([Date]), ((QUARTER([Date])-1)*3)+1, 1),
    "Year Start Date", DATE(YEAR([Date]), 1, 1),

    -- Relative Calculations
    "Days from Today", DATEDIFF([Date], TODAY(), DAY),
    "Months from Today", DATEDIFF([Date], TODAY(), MONTH),

    -- Holiday Detection
    "Is Holiday",
        (MONTH([Date]) = 1 && DAY([Date]) = 1) ||
        (MONTH([Date]) = 7 && DAY([Date]) = 4) ||
        (MONTH([Date]) = 12 && DAY([Date]) = 25) ||
        (MONTH([Date]) = 11 && WEEKDAY([Date]) = 5 && DAY([Date]) >= 22 &&
DAY([Date]) <= 28) ||
        (MONTH([Date]) = 9 && WEEKDAY([Date]) = 2 && DAY([Date]) <= 7),

    -- Business Day (calculated inline, not referencing other columns)
    "Is Business Day", NOT(WEEKDAY([Date],2) > 5) && NOT(
        (MONTH([Date]) = 1 && DAY([Date]) = 1) ||
        (MONTH([Date]) = 7 && DAY([Date]) = 4) ||
        (MONTH([Date]) = 12 && DAY([Date]) = 25) ||
        (MONTH([Date]) = 11 && WEEKDAY([Date]) = 5 && DAY([Date]) >= 22 &&
DAY([Date]) <= 28) ||
        (MONTH([Date]) = 9 && WEEKDAY([Date]) = 2 && DAY([Date]) <= 7)
    ),

    -- Relative Periods
    "Relative Period",
        VAR DaysFromToday = DATEDIFF([Date], TODAY(), DAY)
        RETURN
        SWITCH(TRUE(),
            DaysFromToday >= -30 && DaysFromToday <= 0, "Last 30 Days",
            DaysFromToday >= -90 && DaysFromToday <= 0, "Last 90 Days",
            DaysFromToday >= -365 && DaysFromToday <= 0, "Last 12 Months",
            DaysFromToday > 0 && DaysFromToday <= 30, "Next 30 Days",
            DaysFromToday > 0 && DaysFromToday <= 90, "Next 90 Days",
            BLANK()
        ),

    -- Time Intelligence Flags
```

```
    "Is Current Year", YEAR([Date]) = YEAR(TODAY()),
    "Is Current Quarter", YEAR([Date]) = YEAR(TODAY()) && QUARTER([Date]) =
QUARTER(TODAY()),
    "Is Current Month", YEAR([Date]) = YEAR(TODAY()) && MONTH([Date]) =
MONTH(TODAY()),
    "Is Previous Year", YEAR([Date]) = YEAR(TODAY()) - 1,

    -- Season Indicators
    "Season", SWITCH(TRUE(),
        MONTH([Date]) >= 3 && MONTH([Date]) <= 5, "Spring",
        MONTH([Date]) >= 6 && MONTH([Date]) <= 8, "Summer",
        MONTH([Date]) >= 9 && MONTH([Date]) <= 11, "Fall",
        "Winter"
    ),

    -- Sort Columns
    "Month Sort", MONTH([Date]),
    "Day Sort", WEEKDAY([Date]),
    "Quarter Sort", QUARTER([Date])
)
```

// M Language Power Query Date Table (Dynamic Range, Business Days, ISO/US Week, Seasons, Formatting)

```
// M Language Power Query Date Table (Dynamic Range, Business Days, ISO/US
Week, Seasons, Formatting)
let
    // Get min/max date from Orders table (replace with your table name)
    Source =
Excel.Workbook(File.Contents("d:\Neokai\Github_PBI\AgenticPowerBI\global_sal
es.xlsx"), null, true),
    Orders_Sheet = Source{[Item="Orders",Kind="Sheet"]}[Data],
    Orders = Table.PromoteHeaders(Orders_Sheet, [PromoteAllScalars=true]),
    OrdersWithDate = Table.TransformColumns(Orders, {{"Order Date", each
Date.From(Number.From(_)), type date}}),
    MinDate = List.Min(OrdersWithDate[Order Date]),
    MaxDate = List.Max(OrdersWithDate[Order Date]),

    // Generate continuous date list
    DateList = List.Dates(MinDate, Duration.Days(MaxDate - MinDate) + 1,
#duration(1,0,0,0)),
    DateTable = Table.FromList(DateList, Splitter.SplitByNothing(),
{"Date"}),

    // Add standard columns
    AddDateID = Table.AddColumn(DateTable, "Date ID", each
Date.ToText([Date], "yyyyMMdd"), type text),
    AddYear = Table.AddColumn(AddDateID, "Year", each Date.Year([Date]),
```

```
Int64.Type),
    AddQuarter = Table.AddColumn(AddYear, "Quarter", each
Date.QuarterOfYear([Date]), Int64.Type),
    AddMonth = Table.AddColumn(AddQuarter, "Month", each Date.Month([Date]),
Int64.Type),
    AddMonthName = Table.AddColumn(AddMonth, "Month Name", each
Date.ToText([Date], "MMMM"), type text),
    AddWeek = Table.AddColumn(AddMonthName, "Week", each
Date.WeekOfYear([Date]), Int64.Type),
    AddISOWeek = Table.AddColumn(AddWeek, "ISO Week", each
Date.WeekOfYear([Date], Day.Monday), Int64.Type),
    AddDayOfWeek = Table.AddColumn(AddISOWeek, "Day of Week", each
Date.DayOfWeek([Date]), Int64.Type),
    AddDayName = Table.AddColumn(AddDayOfWeek, "Day Name", each
Date.ToText([Date], "dddd"), type text),

    // Fiscal columns (Fiscal year starts April 1st)
    AddFiscalYear = Table.AddColumn(AddDayName, "Fiscal Year", each if
Date.Month([Date]) >= 4 then Date.Year([Date]) + 1 else Date.Year([Date]),
Int64.Type),
    AddFiscalQuarter = Table.AddColumn(AddFiscalYear, "Fiscal Quarter", each
Number.Mod(Date.QuarterOfYear([Date]) + 2, 4) + 1, Int64.Type),
    AddFiscalMonth = Table.AddColumn(AddFiscalQuarter, "Fiscal Month", each
if Date.Month([Date]) >= 4 then Date.Month([Date]) - 3 else
Date.Month([Date]) + 9, Int64.Type),

    // Weekday/Weekend/Business Day
    AddIsWeekend = Table.AddColumn(AddFiscalMonth, "Is Weekend", each let d
= Date.DayOfWeek([Date], Day.Monday) in d >= 5, type logical),
    AddIsBusinessDay = Table.AddColumn(AddIsWeekend, "Is Business Day", each
not [Is Weekend], type logical),

    // US Holidays (simplified)
    AddIsHoliday = Table.AddColumn(AddIsBusinessDay, "Is Holiday", each
        let
            m = Date.Month([Date]),
            d = Date.Day([Date])
        in
            (m = 1 and d = 1) or // New Year's Day
            (m = 7 and d = 4) or // Independence Day
            (m = 12 and d = 25), // Christmas
        type logical
    ),

    // Season indicators
    AddSeason = Table.AddColumn(AddIsHoliday, "Season", each
        let
            m = Date.Month([Date])
        in
            if m = 12 or m = 1 or m = 2 then "Winter"
```

```
            else if m = 3 or m = 4 or m = 5 then "Spring"
            else if m = 6 or m = 7 or m = 8 then "Summer"
            else "Fall",
        type text
    ),

    // Date formatting variations
    AddShortDate = Table.AddColumn(AddSeason, "Short Date", each
Date.ToText([Date], "MM/dd/yyyy"), type text),
    AddLongDate = Table.AddColumn(AddShortDate, "Long Date", each
Date.ToText([Date], "dddd, MMMM dd, yyyy"), type text),

    // Week Start, Month Start, Quarter Start
    AddWeekStart = Table.AddColumn(AddLongDate, "Week Start Date", each
Date.AddDays([Date], -Date.DayOfWeek([Date], Day.Monday)), type date),
    AddMonthStart = Table.AddColumn(AddWeekStart, "Month Start Date", each
Date.StartOfMonth([Date]), type date),
    AddQuarterStart = Table.AddColumn(AddMonthStart, "Quarter Start Date",
each Date.StartOfQuarter([Date]), type date),

    // Days from Today, Relative Period
    AddDaysFromToday = Table.AddColumn(AddQuarterStart, "Days from Today",
each Duration.Days([Date] - Date.From(DateTime.LocalNow())), Int64.Type),
    AddRelativePeriod = Table.AddColumn(AddDaysFromToday, "Relative Period",
each
        if [Date] >= Date.AddDays(Date.From(DateTime.LocalNow()), -29) and
[Date] <= Date.From(DateTime.LocalNow()) then "Last 30 Days"
        else if [Date] >= Date.AddDays(Date.From(DateTime.LocalNow()), -89)
and [Date] <= Date.From(DateTime.LocalNow()) then "Last 90 Days"
        else null, type text
    ),

    // Time Intelligence: Previous Year, Quarter, Month, YTD, QTD, MTD, Same
Period Last Year
    AddPrevYear = Table.AddColumn(AddRelativePeriod, "Previous Year", each
Date.AddYears([Date], -1), type date),
    AddPrevQuarter = Table.AddColumn(AddPrevYear, "Previous Quarter", each
Date.AddQuarters([Date], -1), type date),
    AddPrevMonth = Table.AddColumn(AddPrevQuarter, "Previous Month", each
Date.AddMonths([Date], -1), type date),
    AddYTD = Table.AddColumn(AddPrevMonth, "YTD", each [Date] >=
Date.StartOfYear([Date]) and [Date] <= Date.From(DateTime.LocalNow()), type
logical),
    AddQTD = Table.AddColumn(AddYTD, "QTD", each [Date] >=
Date.StartOfQuarter([Date]) and [Date] <= Date.From(DateTime.LocalNow()),
type logical),
    AddMTD = Table.AddColumn(AddQTD, "MTD", each [Date] >=
Date.StartOfMonth([Date]) and [Date] <= Date.From(DateTime.LocalNow()), type
logical),
    AddSamePeriodLastYear = Table.AddColumn(AddMTD, "Same Period Last Year",
```

```
each Date.AddYears([Date], -1), type date)
in
    AddSamePeriodLastYear
```

3. Data Quality Checks

- Copilot suggests Power Query functions that flag missing or duplicate records.
- Example: Auto-generate M code for “Null check on Revenue” column.

Prompts

```
"Create Power Query M language functions for data quality checks in Power BI
with these requirements:

1. Missing Value Detection:
   - Flag null/blank values in critical columns (Sales, Profit, Customer ID,
Product ID)
   - Count missing values by column
   - Create data quality summary report
   - Add conditional columns showing data completeness percentage

2. Duplicate Record Identification:
   - Detect duplicate Order IDs
   - Find duplicate customer records
   - Identify potential duplicate products
   - Create duplicate flags and counts

3. Data Validation Rules:
   - Sales amount validation (negative values, outliers)
   - Date range validation (Order Date vs Ship Date logic)
   - Text field validation (consistent formatting, valid entries)
   - Numeric range validation (quantities, discounts within expected ranges)

4. Automated Quality Scoring:
   - Overall data quality score (0-100%)
   - Quality score by table/column
   - Traffic light indicators (Red/Yellow/Green)
   - Quality trend tracking over time

5. Simple M Language Functions:
   - Easy to copy-paste into Power Query
   - No complex coding required
   - Works with existing global sales data structure
   - Generates actionable quality reports

Output ready-to-use M language code for Power Query that can be applied
immediately to detect and flag data quality issues in the global sales
dataset."
```

Power Query M Functions for Data Quality Checks on Global Sales Data

```
let
    // Simple data quality function that actually works
    AddDataQualityChecks = (SourceTable as table) =>
    let
        // Step 1: Add missing value flags
        AddMissingFlags = Table.AddColumn(
            Table.AddColumn(
                Table.AddColumn(
                    Table.AddColumn(SourceTable,
                        "Sales_Missing", each [Sales] = null or [Sales] =
"", type logical),
                    "Profit_Missing", each [Profit] = null or [Profit] = "",
type logical),
                "CustomerID_Missing", each [Customer ID] = null or [Customer
ID] = "", type logical),
            "ProductID_Missing", each [Product ID] = null or [Product ID] =
"", type logical),

        // Step 2: Add validation flags
        AddValidations = Table.AddColumn(
            Table.AddColumn(
                Table.AddColumn(
                    Table.AddColumn(
                        Table.AddColumn(AddMissingFlags,
                            "Sales_Valid", each [Sales] >= 0 and [Sales] <=
50000, type logical),
                        "Profit_Valid", each [Profit] >= -5000 and [Profit]
<= 25000, type logical),
                    "Quantity_Valid", each [Quantity] >= 1 and [Quantity] <=
100, type logical),
                "Discount_Valid", each [Discount] >= 0 and [Discount] <= 1,
type logical),
            "Date_Valid", each [Order Date] <= [Ship Date], type logical),

        // Step 3: Calculate quality score per row
        AddQualityScore = Table.AddColumn(AddValidations, "Quality_Score",
            each
                let
                    checks = {
                        not [Sales_Missing],
                        not [Profit_Missing],
                        not [CustomerID_Missing],
                        not [ProductID_Missing],
                        [Sales_Valid],
                        [Profit_Valid],
```

```
                        [Quantity_Valid],
                        [Discount_Valid],
                        [Date_Valid]
                    },
                    passed = List.Count(List.Select(checks, each _ = true)),
                    total = List.Count(checks)
                in
                    Number.Round((passed / total) * 100, 0),
            Int64.Type),

        // Step 4: Add traffic light indicator
        AddIndicator = Table.AddColumn(AddQualityScore, "Quality_Flag",
            each if [Quality_Score] >= 90 then "Good"
                 else if [Quality_Score] >= 70 then "Warning"
                 else "Poor", type text)
    in
        AddIndicator
in
    AddDataQualityChecks
```

4. Automating Data Refresh Scripts

- Generate PowerShell/Azure CLI scripts for scheduled refresh of Power BI datasets.
- Example: Refresh dataset after nightly SQL ETL completion.

B. Data Modeling & DAX

1. Automated DAX Measures

- Copilot generates common DAX calculations (YoY growth, Variance %, Moving Average).
- Example: "Write DAX for 12-month rolling sales" → Copilot provides optimized formula.

Prompt

```
"Generate common DAX measures for Power BI sales analysis with these
requirements:

1. Time Intelligence Measures:
   - YoY (Year-over-Year) sales growth percentage
   - QoQ (Quarter-over-Quarter) growth percentage
   - 12-month rolling average sales
   - Same period last year comparisons
   - YTD (Year-to-Date) vs Previous YTD
```

```
2. Statistical Measures:
   - Moving averages (3-month, 6-month, 12-month)
   - Variance percentage calculations
   - Standard deviation of sales
   - Trend analysis (growth acceleration/deceleration)
   - Seasonality indicators

3. Business Performance Measures:
   - Top N customers/products (dynamic ranking)
   - Market share calculations
   - Customer retention rates
   - Average order value trends
   - Profit margin analysis

4. Optimized DAX Patterns:
   - Use CALCULATE, DATEADD, SAMEPERIODLASTYEAR functions
   - Proper error handling with DIVIDE function
   - Performance-optimized formulas for large datasets
   - Context transition best practices

Output ready-to-use DAX measures that can be copied directly into Power BI
for immediate use with global sales data structure."
```

Time Intelligence Measures

```
// YoY Sales Growth % - ✅ CORRECT
YoY Sales Growth % =
VAR SalesThisYear = CALCULATE([Total Sales], DATESYTD('Orders'[Order Date]))
VAR SalesLastYear = CALCULATE([Total Sales],
DATESYTD(SAMEPERIODLASTYEAR('Orders'[Order Date])))
RETURN DIVIDE(SalesThisYear - SalesLastYear, SalesLastYear, 0)

// QoQ Sales Growth % - ❌ FIXED
QoQ Sales Growth % =
VAR SalesThisQ = CALCULATE([Total Sales], DATESQTD('Orders'[Order Date]))
VAR SalesPrevQ = CALCULATE([Total Sales], DATEADD(DATESQTD('Orders'[Order
Date]), -1, QUARTER))
RETURN DIVIDE(SalesThisQ - SalesPrevQ, SalesPrevQ, 0)

// 12M Rolling Avg Sales - ❌ IMPROVED
12M Rolling Avg Sales =
VAR Last12Months = DATESINPERIOD('Orders'[Order Date],
LASTDATE('Orders'[Order Date]), -12, MONTH)
RETURN
CALCULATE(
    AVERAGEX(
        SUMMARIZE('Orders', 'Orders'[Order Date]),
```

```
        [Total Sales]
    ),
    Last12Months
)
```

Statistical Measures

```
3M Moving Avg Sales =
CALCULATE(
    AVERAGEX(
        DATESINPERIOD('Orders'[Order Date], LASTDATE('Orders'[Order Date]),
-3, MONTH),
        [Total Sales]
    )
)


6M Moving Avg Sales =
CALCULATE(
    AVERAGEX(
        DATESINPERIOD('Orders'[Order Date], LASTDATE('Orders'[Order Date]),
-6, MONTH),
        [Total Sales]
    )
)



Variance % =
VAR Actual = [Total Sales]
VAR Target = 1000000  // Set your actual target or create Target Sales
measure
RETURN DIVIDE(Actual - Target, Target, 0)

Sales Std Dev =
STDEVX.S(
    SUMMARIZE('Orders', 'Orders'[Order Date], "@DailySales", [Total Sales]),
    [@DailySales]
)



Trend Acceleration =
VAR PrevGrowth = CALCULATE([YoY Sales Growth %], DATEADD('Orders'[Order
Date], -1, YEAR))
RETURN IF(ISBLANK(PrevGrowth), BLANK(), [YoY Sales Growth %] - PrevGrowth)
```

```
Seasonality Indicator =
VAR MonthAvg =
    CALCULATE(
        AVERAGE('Orders'[Sales]),
        ALL('Orders'),
        MONTH('Orders'[Order Date]) = MONTH(TODAY())
    )
VAR YearAvg =
    CALCULATE(
        AVERAGE('Orders'[Sales]),
        ALL('Orders')
    )
RETURN
    DIVIDE(MonthAvg, YearAvg, 0)
```

Business Performance Measures

```
Top 10 Customer Flag =
VAR CustomerRank = RANKX(ALL('Orders'[Customer Name]), [Total Sales],, DESC)
RETURN IF(CustomerRank <= 10, [Total Sales], BLANK())

Top 10 Product Flag =
VAR ProductRank = RANKX(ALL('Orders'[Product Name]), [Total Profit],, DESC)
RETURN IF(ProductRank <= 10, [Total Profit], BLANK())

Market Share % =
DIVIDE([Total Sales], CALCULATE([Total Sales], ALL('Orders')), 0)

Customer Retention Rate =
VAR PrevCustomers = CALCULATETABLE(VALUES('Orders'[Customer ID]),
DATEADD('Orders'[Order Date], -1, YEAR))
VAR CurrentCustomers = VALUES('Orders'[Customer ID])
RETURN DIVIDE(COUNTROWS(INTERSECT(PrevCustomers, CurrentCustomers)),
COUNTROWS(PrevCustomers), 0)

Average Order Value Trend =
CALCULATE(
    AVERAGEX(VALUES('Orders'[Order ID]), [Total Sales]),
    DATESINPERIOD('Orders'[Order Date], LASTDATE('Orders'[Order Date]), -12,
MONTH)
)
```

2. Relationship Suggestions

- Copilot scans tables and proposes model relationships based on matching keys.
- Example: CustomerID in Sales linked with Customer table automatically.

Done Earlier

3. Dynamic Variance Analysis

- Generate DAX for PY vs CY, Budget vs Actual, Plan vs Forecast.
- FP&A teams save hours of coding time.

Prompt

```
"Create DAX measures for financial variance analysis in Power BI with these
requirements:

1. Time Period Comparisons:
   - Current Year vs Previous Year (CY vs PY)
   - Current Quarter vs Previous Quarter (CQ vs PQ)
   - Current Month vs Previous Month (CM vs PM)
   - Same period last year comparisons
   - Variance amounts and percentages for each

2. Budget vs Actual Analysis:
   - Actual vs Budget variance (amount and %)
   - YTD Actual vs YTD Budget
   - Favorable/Unfavorable variance indicators
   - Budget variance by time periods (monthly, quarterly)

3. Plan vs Forecast Comparisons:
   - Plan vs Latest Forecast variance
   - Forecast accuracy measures
   - Rolling forecast vs static plan
   - Forecast revision tracking

4. Dynamic Variance Formatting:
   - Positive/negative variance indicators
   - Traffic light formatting (Red/Yellow/Green)
   - Variance thresholds (material vs immaterial)
   - Automatic favorable/unfavorable classification

5. FP&A Ready Measures:
   - Works with existing sales data structure
   - Simple copy-paste DAX formulas
   - No complex setup required
   - Handles missing budget/plan data gracefully

Output complete DAX measures for variance analysis that FP&A teams can use
immediately for financial reporting and analysis."
```

Solutions

```
// TIME PERIOD COMPARISONS
CY Sales = CALCULATE([Total Sales], YEAR('Orders'[Order Date]) =
YEAR(TODAY()))
PY Sales = CALCULATE([Total Sales], YEAR('Orders'[Order Date]) =
YEAR(TODAY()) - 1)
CY vs PY Variance = [CY Sales] - [PY Sales]
CY vs PY Variance % = DIVIDE([CY vs PY Variance], [PY Sales], 0)

// Quarter Logic
CQ Sales = CALCULATE([Total Sales],
    YEAR('Orders'[Order Date]) = YEAR(TODAY()) &&
    QUARTER('Orders'[Order Date]) = QUARTER(TODAY()))

PQ Sales = CALCULATE([Total Sales],
    DATEADD(DATESQTD('Orders'[Order Date]), -1, QUARTER))

CQ vs PQ Variance = [CQ Sales] - [PQ Sales]
CQ vs PQ Variance % = DIVIDE([CQ vs PQ Variance], [PQ Sales], 0)

// Month Logic
CM Sales = CALCULATE([Total Sales],
    YEAR('Orders'[Order Date]) = YEAR(TODAY()) &&
    MONTH('Orders'[Order Date]) = MONTH(TODAY()))

PM Sales = CALCULATE([Total Sales],
    DATEADD(DATESMTD('Orders'[Order Date]), -1, MONTH))

CM vs PM Variance = [CM Sales] - [PM Sales]
CM vs PM Variance % = DIVIDE([CM vs PM Variance], [PM Sales], 0)

Same Period Last Year Sales = CALCULATE([Total Sales],
SAMEPERIODLASTYEAR('Orders'[Order Date]))
Same Period Variance = [Total Sales] - [Same Period Last Year Sales]
Same Period Variance % = DIVIDE([Same Period Variance], [Same Period Last
Year Sales], 0)

// BUDGET MEASURES (Create dummy targets)
Budget Sales = [Total Sales] * 1.1  // 10% above actual as dummy budget
Actual vs Budget Variance = [Total Sales] - [Budget Sales]
Actual vs Budget Variance % = DIVIDE([Actual vs Budget Variance], [Budget
Sales], 0)

YTD Actual Sales = CALCULATE([Total Sales], DATESYTD('Orders'[Order Date]))
YTD Budget Sales = CALCULATE([Budget Sales], DATESYTD('Orders'[Order Date]))
YTD Actual vs Budget Variance = [YTD Actual Sales] - [YTD Budget Sales]
YTD Actual vs Budget Variance % = DIVIDE([YTD Actual vs Budget Variance],
[YTD Budget Sales], 0)

// VARIANCE INDICATORS (Fixed)
```

```
Variance Indicator =
IF([Actual vs Budget Variance] > 0, "Favorable", "Unfavorable")

Variance Traffic Light =
SWITCH(TRUE(),
    [Actual vs Budget Variance %] >= 0.05, "Green",
    [Actual vs Budget Variance %] < -0.05, "Red",
    "Yellow")
```

4. Scenario & What-If Parameters

- Copilot helps create DAX-driven scenarios (Price +10%, Cost –5%) for sensitivity analysis.

Prompt

```
"Create DAX-driven scenario and what-if parameters for Power BI sensitivity
analysis with these requirements:

1. Price Scenario Analysis:

   - Price increase scenarios (+5%, +10%, +15%, +20%)

   - Price decrease scenarios (-5%, -10%, -15%)

   - Dynamic price adjustment calculations

   - Revenue impact from price changes

2. What-If Parameter Setup:

   - Create Power BI parameter tables

   - Slicer-driven scenario selection

   - Dynamic DAX measures responding to parameter changes

   - Easy-to-use scenario comparison dashboard

Output complete DAX formulas and parameter setup instructions for immediate
implementation in Power BI using existing sales data structure."
```

Solution

```
// Calculate unit price from existing data
Unit Price = DIVIDE([Total Sales], SUM('Orders'[Quantity]), 0)
```

```
// Fixed Adjusted Price calculation
Adjusted Price =
SUMX(
    'Orders',
    ([Unit Price] * 'Orders'[Quantity]) * (1 + SELECTEDVALUE('Price
Adjustment %'[Price Adjustment %], 0))
)

// Or simpler approach using total sales
Adjusted Sales =
[Total Sales] * (1 + SELECTEDVALUE('Price Adjustment %'[Price Adjustment %
Value], 0))

Revenue Impact = [Adjusted Sales] - [Total Sales]

Revenue Impact % = DIVIDE([Revenue Impact], [Total Sales], 0)
```

Setup Steps:

1. **Modeling tab → New parameter → What-if parameter**
2. **Name:** Price Adjustment %
3. **Min:** -0.15, **Max:** 0.20, **Increment:** 0.055
4. **Add the parameter slicer** to your report
5. **Test with cards** showing Adjusted Sales, Revenue Impact

---

C. Report Development & Visualization

1. Automated Report Theme JSON -> DOne

- Copilot generates Power BI JSON theme files (colors, fonts, layouts) aligned with brand guidelines.

Prompt

```
Give me a professional analytics theme that I can use in my Power BI
Dashboard
```

Solutions

```
{
  "name": "Modern Analytics Dashboard Theme",
  "dataColors": [
    "#005B4F",
    "#F2994A",
```

```
    "#EB5757",
    "#56CC9D",
    "#2D9CDB",
    "#BB6BD9",
    "#F2C94C",
    "#6FCF97",
    "#9B51E0",
    "#333333",
    "#27AE60",
    "#FF6F61"
  ],
  "background": "#F3F6FB",
  "foreground": "#1A1A1A",
  "tableAccent": "#005B4F",
  "visualStyles": {
    "*": {
      "*": {
        "background": [{"color": {"solid": {"color": "#FFFFFF"}}}],
        "border": [{"color": {"solid": {"color": "#D1D5DB"}}}]
      }
    },
    "card": {
      "*": {
        "background": [{"color": {"solid": {"color": "#FFFFFF"}}}],
        "border": [{"color": {"solid": {"color": "#D1D5DB"}}}],
        "dropShadow": [{"show": true, "color": {"solid": {"color":
"#00000020"}}}]
      }
    },
    "tableEx": {
      "*": {
        "background": [{"color": {"solid": {"color": "#FFFFFF"}}}],
        "border": [{"color": {"solid": {"color": "#D1D5DB"}}}]
      }
    },
    "slicer": {
      "*": {
        "background": [{"color": {"solid": {"color": "#FFFFFF"}}}],
        "border": [{"color": {"solid": {"color": "#D1D5DB"}}}]
      }
    }
  },
  "textClasses": {
    "callout": {
      "fontSize": 45,
      "fontFace": "Segoe UI Light",
      "color": "#005B4F"
    },
    "title": {
      "fontSize": 20,
```

```
      "fontFace": "Segoe UI Semibold",
      "color": "#005B4F"
    },
    "header": {
      "fontSize": 16,
      "fontFace": "Segoe UI Semibold",
      "color": "#1A1A1A"
    },
    "label": {
      "fontSize": 13,
      "fontFace": "Segoe UI",
      "color": "#333333"
    }
  }
}
```

2. Dynamic Tooltip Pages

- Copilot suggests DAX & bookmarks setup for interactive tooltip pages with drillthrough insights.

Prompt:

```
"Create dynamic tooltip pages setup for Power BI with these requirements:

1. Tooltip Page Configuration:
   - Create dedicated tooltip report pages
   - Configure page size for tooltip display (320x240 pixels)
   - Set up proper visual layout for small tooltip space
   - Enable tooltip functionality in visual settings

2. Context-Aware Tooltip Content:
   - Customer detail tooltips (total orders, lifetime value, last order
date)
   - Product detail tooltips (profit margin, sales rank, inventory status)
   - Regional performance tooltips (vs target, growth trends, top
performers)
   - Time-based tooltips (YoY comparison, seasonal trends, forecasts)

3. Interactive Tooltip Features:
   - Drill-through actions from tooltips
   - Bookmark navigation from tooltip content
   - Cross-highlight functionality
   - Dynamic tooltip content based on filters

Output complete setup instructions, DAX measures, and configuration steps
for immediate tooltip implementation."
```

Solutions

```
# Dynamic Tooltip Pages Setup Guide

## Phase 2: Tooltip-Specific DAX Measures

### Customer Tooltip Measures


// Customer lifetime value for tooltip
Customer Lifetime Value =
CALCULATE([Total Sales], ALL('Orders'[Order Date]))

// Last order date for customer
Last Order Date =
FORMAT(
    CALCULATE(MAX('Orders'[Order Date]), ALL('Orders'[Order Date])),
    "MM/DD/YYYY"
)

// Customer order count
Customer Order Count =
CALCULATE(DISTINCTCOUNT('Orders'[Order ID]), ALL('Orders'[Order Date]))

// Customer status indicator
Customer Status =
VAR DaysSinceLastOrder = DATEDIFF(CALCULATE(MAX('Orders'[Order Date]),
ALL('Orders'[Order Date])), TODAY(), DAY)
RETURN
SWITCH(
    TRUE(),
    DaysSinceLastOrder <= 90, "Active",
    DaysSinceLastOrder <= 365, "At Risk",
    "Inactive"
)


### Product Tooltip Measures


// Product sales rank for tooltip
Product Sales Rank =
RANKX(ALL('Orders'[Product Name]), [Total Sales],, DESC)

// Product profit margin for tooltip
Product Profit Margin =
DIVIDE([Total Profit], [Total Sales], 0)
```

```
// Product performance indicator
Product Performance =
VAR Rank = [Product Sales Rank]
RETURN
SWITCH(
    TRUE(),
    Rank <= 10, "Top Performer",
    Rank <= 50, "Good",
    Rank <= 100, "Average",
    "Below Average"
)

// Product sales trend (3-month)
Product 3M Trend =
VAR Current3M = CALCULATE([Total Sales], DATESINPERIOD('Orders'[Order Date],
LASTDATE('Orders'[Order Date]), -3, MONTH))
VAR Previous3M = CALCULATE([Total Sales], DATESINPERIOD('Orders'[Order
Date], LASTDATE('Orders'[Order Date]), -6, MONTH))
RETURN
DIVIDE(Current3M - Previous3M, Previous3M, 0)
```

Main Chart: Horizontal Bar Chart

- Y-axis: Customer Name
- X-axis: Total Sales
- Sort by Total Sales (descending)
- Show top 15-20 customers

Customer Tooltip Page Content:

- Card 1: Customer Name (title)
- Card 2: Customer Lifetime Value
- Card 3: Customer Order Count
- Card 4: Last Order Date
- Card 5: Customer Status

Product Tooltip Test Visual:

Main Chart: Scatter Chart

- X-axis: Total Sales
- Y-axis: Total Profit
- Details: Product Name
- Size: Quantity (optional)

Product Tooltip Page Content:

- Card 1: Product Name (title)
- Card 2: Product Sales Rank

- Card 3: Product Profit Margin
- Card 4: Product Performance
- Small line chart: Product 3M Trend

Why These Work Well:

Bar Chart for Customers: Each bar represents a distinct customer, making hover context clear and tooltip assignment straightforward.

Scatter Chart for Products: Each dot represents a product, providing natural hover targets with clear product context for tooltips.

Regional Tooltip Measures

```
// Regional YoY growth for tooltip
Regional YoY Growth =
VAR CurrentYear = CALCULATE([Total Sales], DATESYTD('Orders'[Order Date]))
VAR PreviousYear = CALCULATE([Total Sales],
DATESYTD(SAMEPERIODLASTYEAR('Orders'[Order Date])))
RETURN
DIVIDE(CurrentYear - PreviousYear, PreviousYear, 0)

// Regional vs target (assuming 10% growth target)
Regional vs Target =
VAR Target = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Orders'[Order
Date])) * 1.1
VAR Actual = [Total Sales]
RETURN
DIVIDE(Actual - Target, Target, 0)

// Regional performance status
Regional Status =
VAR Growth = [Regional YoY Growth]
RETURN
SWITCH(
    TRUE(),
    Growth >= 0.15, "Exceeding",
    Growth >= 0.05, "On Track",
    Growth >= 0, "Below Target",
    "Declining"
)
```

Phase 3: Enable Tooltip Functionality

Step 1: Assign Tooltips to Charts

```
1. **Select your main chart** (bar chart, scatter plot, etc.)
2. **Format pane** → **General** → **Tooltips**
3. **Type:** Report page
4. **Page:** Select your tooltip page
5. **Keep default tooltips:** Off (optional)

### Step 2: Configure Tooltip Filters

**For Customer Tooltip:**

- Add Customer ID or Customer Name to tooltip page filters
- Set to "Pass all filters"

**For Product Tooltip:**

- Add Product ID or Product Name to tooltip page filters
- Set to "Pass all filters"

**For Regional Tooltip:**

- Add Region to tooltip page filters
- Set to "Pass all filters"

---
```

3. Drill-Through from Tooltips

1. **Create drill-through target page**
2. **Add drill-through fields** to target page
3. **On tooltip page:**
   - Add small "Details" button or icon
   - Configure as drill-through action

```
**Drill-Through from Tooltips:**

**Step 1: Create Drill-Through Target Page**

1. Add new report page
2. Name it "Customer Details"
3. Add detailed visuals (customer sales over time, product breakdown, etc.)

**Step 2: Set Up Drill-Through**

1. On Customer Details page, go to **Visualizations pane**
2. **Drill-through section** → Drag **Customer Name** field
```

```
3. This enables drill-through to this page

**Step 3: Add Button to Tooltip**

1. On your tooltip page, add **Button visual**
2. Button text: "View Details"
3. **Button properties:**
    - Type: Drill-through
    - Destination: Customer Details page
4. Make button small to fit tooltip space
```

Bookmark Navigation

// Bookmark trigger measure
Navigate to Details =
IF(HASONEVALUE('Orders'[Customer Name]), "View Details", BLANK())

1. **Create bookmark** for detailed view
2. **Add button** to tooltip with bookmark action
3. **Use conditional formatting** to show/hide button

```
**Step 1: Create the DAX Measure**

dax

```dax
Navigate to Details =
IF(HASONEVALUE('Orders'[Customer Name]), "View Details", BLANK())
```

Step 2: Create Bookmark

1. Set up your report in desired state (specific filters, page, etc.)
2. **View tab** → **Bookmarks** → **Add bookmark**
3. Name it "Customer Detail View"

Step 3: Add Button with Bookmark Action

1. On tooltip page, add **Button visual**
2. **Action settings:**
    - Type: Bookmark
    - Bookmark: Customer Detail View
3. **Conditional formatting:**
    - Based on "Navigate to Details" measure
    - Only show when measure has value
```

3. KPI Cards Automation

- Auto-generate multiple KPI visuals (Revenue, EBITDA, NPS, etc.) using Copilot templates.

Prompt:

```
"Create automated KPI card templates for Power BI with these requirements:

1. Essential Business KPIs:
   - Revenue metrics (Total Revenue, Revenue Growth %, Revenue vs Target)
   - Profitability metrics (Gross Profit, Profit Margin %, Net Profit)
   - Operational metrics (Order Count, Average Order Value, Customer Count)
   - Performance indicators (YoY Growth, QoQ Growth, Market Share)

2. KPI Card Design Templates:
   - Consistent formatting and layout
   - Color-coded performance indicators (green/red/yellow)
   - Trend arrows and variance indicators
   - Conditional formatting based on thresholds

3. KPI Card Configuration:
   - Standard card visual settings
   - Consistent fonts, colors, and sizing
   - Automated conditional formatting rules
   - Template for easy replication

Output ready-to-use DAX measures and KPI card setup instructions for
immediate implementation with global sales data.
```

Solution

```
**Use These Existing Measures:**

- [Total Sales] → Revenue KPI
- [YoY Sales Growth %] → Growth KPI
- [Profit Margin %] → Profitability KPI
- [Average Order Value Trend] → AOV KPI
- [Customer Retention Rate] → Customer KPI

// Trend Arrow (reuses existing growth measure)
Growth Trend Arrow =
SWITCH(
    TRUE(),
    [YoY Sales Growth %] > 0.05, "▲",
    [YoY Sales Growth %] < -0.05, "▼",
```

```
    "►"
)

// KPI Status Color
KPI Status Color =
SWITCH(
    TRUE(),
    [YoY Sales Growth %] > 0.05, "#388E3C",
    [YoY Sales Growth %] < -0.05, "#D32F2F",
    "#FBC02D"
)
```

KPI Card Setup:

1. **Card 1:** [Total Sales] + [Growth Trend Arrow]
2. **Card 2:** [YoY Sales Growth %] + conditional color
3. **Card 3:** [Profit Margin %] + conditional color
4. **Card 4:** [Customer Retention Rate] + status indicator

Format each card:

- Background color: Use [KPI Status Color]
- Add trend arrow in title or subtitle
- Font: Segoe UI Semibold, size 24

Step 1: Create Card Visual

1. Add Card visual to report
2. Drag [Total Sales] to Fields well
3. Resize card appropriately

Step 2: Add Conditional Background Color

1. Select the card
2. Format pane → Visual → General → Effects → Background
3. Click fx (conditional formatting)
4. Format style: Field value
5. Based on field: [KPI Status Color]
6. Apply

Step 3: Add Trend Arrow to Title

1. Format pane → Visual → General → Title
2. Turn ON title
3. Title text: Click fx → Field value
4. Based on field: [Growth Trend Arrow]

Step 4: Format Data Label

```
1. Format pane → Visual → Data label
2. Font: Segoe UI Semibold
3. Text size: 24
4. Color: White (if using colored backgrounds)
```

4. Paginated Report Scripts

- Copilot writes RDL queries & layouts for pixel-perfect financial reports.

Prompts

```
"Create paginated report scripts for Power BI Report Builder using global
sales data with these requirements:

1. Sales Summary Report Layout:
   - Customer sales breakdown with totals and subtotals
   - Product category performance summary
   - Regional sales analysis with rankings
   - Time period comparisons (monthly/quarterly)

2. Data Queries for Global Sales:
   - Extract from Orders table (Sales, Profit, Customer, Product, Region,
Date)
   - Group by Customer, Product, Region for summary sections
   - Calculate totals, subtotals, and percentages
   - Sort by sales amounts and dates

3. Report Formatting:
   - Professional table layouts with proper alignment
   - Headers and footers with report titles and page numbers
   - Subtotal and grand total rows with bold formatting
   - Consistent fonts and spacing throughout

4. Parameters:
   - Date range selector (start date, end date)
   - Region filter dropdown
   - Customer segment filter
   - Export format options

Output RDL template and SQL queries specifically for the global sales
dataset structure with Orders, Returns, and People tables."
```

Solutions

```
-- SQL Queries for Paginated Report DataSets

-- 1. Customer Sales Breakdown
SELECT
    o.[Customer ID],
    o.[Customer Name],
    o.[Segment],
    SUM(o.[Sales]) AS TotalSales,
    SUM(o.[Profit]) AS TotalProfit,
    COUNT(DISTINCT o.[Order ID]) AS OrderCount
FROM Orders o
WHERE o.[Order Date] BETWEEN @StartDate AND @EndDate
  AND (@Region IS NULL OR o.[Region] = @Region)
  AND (@CustomerSegment IS NULL OR o.[Segment] = @CustomerSegment)
GROUP BY o.[Customer ID], o.[Customer Name], o.[Segment]
ORDER BY TotalSales DESC

-- 2. Product Category Performance
SELECT
    o.[Category],
    o.[Subcategory],
    SUM(o.[Sales]) AS CategorySales,
    SUM(o.[Profit]) AS CategoryProfit
FROM Orders o
WHERE o.[Order Date] BETWEEN @StartDate AND @EndDate
  AND (@Region IS NULL OR o.[Region] = @Region)
GROUP BY o.[Category], o.[Subcategory]
ORDER BY CategorySales DESC

-- 3. Regional Sales Analysis
SELECT
    o.[Region],
    o.[Country],
    SUM(o.[Sales]) AS RegionSales,
    SUM(o.[Profit]) AS RegionProfit
FROM Orders o
WHERE o.[Order Date] BETWEEN @StartDate AND @EndDate
GROUP BY o.[Region], o.[Country]
ORDER BY RegionSales DESC

-- 4. Time Period Comparison (Monthly/Quarterly)
SELECT
    YEAR(o.[Order Date]) AS SalesYear,
    MONTH(o.[Order Date]) AS SalesMonth,
    DATEPART(QUARTER, o.[Order Date]) AS SalesQuarter,
    SUM(o.[Sales]) AS PeriodSales,
    SUM(o.[Profit]) AS PeriodProfit
```

```
FROM Orders o
WHERE o.[Order Date] BETWEEN @StartDate AND @EndDate
GROUP BY YEAR(o.[Order Date]), MONTH(o.[Order Date]), DATEPART(QUARTER, o.
[Order Date])
ORDER BY SalesYear, SalesMonth
```

```
### **Step 1: Install Power BI Report Builder**

1. Download from Microsoft (free tool)
2. Install and launch
3. Connect to your Power BI workspace

### **Step 3: Create Report in Report Builder**

1. **New Report** → **Table or Matrix Wizard**
2. **Data Source:** Connect to your data (Power BI dataset or SQL database)
3. **Query:** Paste the corrected SQL
4. **Fields:** Select Customer ID, Customer Name, TotalSales, TotalProfit
5. **Layout:** Choose table layout

### **Step 4: Add Parameters**

1. **Report Data pane** → **Parameters** → **Add**
2. Create: StartDate, EndDate, Region, CustomerSegment
3. Set data types and default values


### **Step 5: Format the Report**

1. **Headers:** Bold, larger font
2. **Numbers:** Currency formatting for sales/profit
3. **Totals:** Add subtotal and grand total rows
4. **Page setup:** Headers, footers, page numbers
```

D. Governance & Security

1. Row-Level Security (RLS) Rules -> Done

- Copilot generates DAX filter expressions for RLS roles (Region Manager sees only their region).

2. Automated Deployment Scripts

- Write YAML pipelines (GitHub Actions) for automated Power BI deployment across Dev, Test, Prod.
- **AND**

2. CI/CD for Power BI

Copilot generates scripts for automated deployment of PBIX files, dataset refresh, and version control using GitHub.

Prompt

```
"Create simple GitHub Actions pipeline for Power BI deployment with these
requirements:

1. Basic Pipeline Structure:
   - Deploy .pbix files from GitHub repository to Power BI workspace
   - Single workspace deployment (your current workspace)
   - Trigger on commits to main branch
   - Use Power BI REST API for uploads

2. Simple Configuration:
   - PowerShell scripts for Power BI API calls
   - GitHub secrets for credentials (service principal)
   - Basic error handling and notifications
   - Works with existing global_sales.pbix file

3. Minimal Setup:
   - One YAML workflow file
   - One PowerShell deployment script
   - Environment variables for workspace ID
   - Basic success/failure notifications

4. Current Repository Structure:
   - Works with D:\Neokai\Github_PBI\AgenticPowerBI folder
   - Deploys to your existing Power BI workspace
   - Handles .pbix file updates and dataset refresh
   - Simple commit-triggered deployment

Output basic YAML workflow and PowerShell script for immediate use with your
current Power BI setup and repository structure."
```

Solution

File 1: `.github/workflows/deploy-powerbi.yml`

```
name: Power BI PBIX Deployment
on:
  push:
    branches:
      - main
jobs:
  deploy-powerbi:
    runs-on: windows-latest
    env:
      PBI_WORKSPACE_ID: ${{ secrets.PBI_WORKSPACE_ID }}
      PBI_CLIENT_ID: ${{ secrets.PBI_CLIENT_ID }}
      PBI_CLIENT_SECRET: ${{ secrets.PBI_CLIENT_SECRET }}
      PBI_TENANT_ID: ${{ secrets.PBI_TENANT_ID }}
    steps:
      - name: Checkout repository
        uses: actions/checkout@v3
      - name: Deploy PBIX to Power BI Workspace
        shell: pwsh
        run: |
          ./scripts/deploy-powerbi.ps1 -WorkspaceId $env:PBI_WORKSPACE_ID -
PbixPath "./global_sales.pbix" -ClientId $env:PBI_CLIENT_ID -ClientSecret
$env:PBI_CLIENT_SECRET -TenantId $env:PBI_TENANT_ID
      - name: Notify Success
        if: success()
        run: echo "Power BI deployment succeeded!"
      - name: Notify Failure
        if: failure()
        run: echo "Power BI deployment failed!"
```

File 2: `scripts/deploy-powerbi.ps1`

```
param(
    [string]$WorkspaceId,
    [string]$PbixPath,
    [string]$ClientId,
    [string]$ClientSecret,
    [string]$TenantId
)

Write-Host "Authenticating to Power BI REST API..."
$body = @{
    grant_type    = "client_credentials"
    client_id     = $ClientId
    client_secret = $ClientSecret
    resource      = "https://analysis.windows.net/powerbi/api"
}

$tokenResponse = Invoke-RestMethod -Uri
```

```
"https://login.microsoftonline.com/$TenantId/oauth2/token" -Method Post -
Body $body
$accessToken = $tokenResponse.access_token

if (-not $accessToken) {
    Write-Error "Failed to authenticate to Power BI API."
    exit 1
}

Write-Host "Uploading PBIX file to workspace..."
$headers = @{
    "Authorization" = "Bearer $accessToken"
}

# Fixed file upload method
$form = @{
    file = Get-Item -Path $PbixPath
}

$uri = "https://api.powerbi.com/v1.0/myorg/groups/$WorkspaceId/imports?
datasetDisplayName=global_sales"

try {
    $response = Invoke-RestMethod -Uri $uri -Headers $headers -Method Post -
Form $form
    Write-Host "PBIX file uploaded successfully."
    Write-Host "Import ID: $($response.id)"
}
catch {
    Write-Error "PBIX upload failed: $_"
    exit 1
}
```

```
### **Step 2: Set Up GitHub Secrets**

1. **GitHub repo** → **Settings** → **Secrets and variables** → **Actions**
2. **Add these secrets:**
    - `PBI_WORKSPACE_ID`: Your workspace GUID
    - `PBI_CLIENT_ID`: Service principal app ID
    - `PBI_CLIENT_SECRET`: Service principal secret
    - `PBI_TENANT_ID`: Your Azure tenant ID

### **Step 3: Create Service Principal**

1. **Azure Portal** → **App registrations** → **New registration**
2. **Note Client ID and Tenant ID**
3. **Certificates & secrets** → **New client secret**
4. **API permissions** → **Power BI Service** → **Dataset.ReadWrite.All**
```

```
### **Step 4: Test Deployment**

1. **Commit files** to main branch
2. **Check Actions tab** for pipeline execution
3. **Verify deployment** in Power BI workspace
```

E. Finance & FP&A Specific

1. Automated Financial Ratios -> Already made too many measures, would look the same

- Copilot writes DAX measures for EPS, ROA, Debt/Equity, Liquidity Ratios, etc.

F. DevOps & Advanced Automation

1. Automated Dataset Documentation

Copilot writes data dictionary docs (columns, measures, transformations) from the PBIX model.

Prompt

```
Based on complete Power BI report `agentic_demo.pbip` and all of its
associated file in our folder and our `global_sales.xlsx` dataset. Please
write the data dictionary docs (columns, measures, transformations) from the
PBIX model.
```

Solution

```
# Power BI Data Dictionary: agentic_demo.pbip & global_sales.xlsx

---

## **1. Orders Table**

| Column Name         | Data Type   | Description
| Transformations / Notes                        |
|---------------------|-------------|-----------------------------------
--------|-----------------------------------------------|
| TransactionID       | Text        | Unique transaction identifier
```

| Cleaned, deduplicated |
| OrderID | Text | Unique order identifier |
| Used for joins, duplicate check |
| Order Date | Date | Date order was placed |
| Converted from Excel serial, formatted |
| Ship Date | Date | Date order was shipped |
| Converted from Excel serial, formatted |
| CustomerID | Text | Unique customer identifier |
| Cleaned, deduplicated |
| Customer Name | Text | Customer full name |
| Trimmed, standardized |
| ProductID | Text | Unique product identifier |
| Cleaned, deduplicated |
| Product Name | Text | Product name |
| Trimmed, standardized |
| Category | Text | Product category |
| Standardized |
| Subcategory | Text | Product subcategory |
| Standardized |
| Region | Text | Sales region |
| Standardized |
| Country | Text | Country of sale |
| Standardized |
| State | Text | State/province |
| Standardized |
| City | Text | City |
| Standardized |
| Sales | Decimal | Sales amount |
| Nulls handled, type enforced |
| Profit | Decimal | Profit amount |
| Nulls handled, type enforced |
| Quantity | Integer | Quantity sold |
| Nulls handled, type enforced |
| Discount | Decimal | Discount percent |
| Nulls handled, type enforced |
| SalespersonID | Text | Salesperson identifier |
| Joined to People table |
| Sales Channel | Text | Channel of sale |
| Standardized |
| Customer Segment | Text | Segment (Consumer, Corporate, etc.) |
| Standardized |
| Priority | Text | Order priority |
| Standardized |
| Status | Text | Order status |
| Standardized |
| Delivery Days | Integer | Ship Date - Order Date |
| Calculated column |
| Profit Margin % | Decimal | Profit / Sales |
| Calculated column |
| Sales Category | Text | Sales bucket (Low, Medium, High, Very

```
High)    | Calculated column                          |
| Customer Tier       | Text       | Bronze/Silver/Gold/Platinum (by sales)
| Calculated column                          |
| Returned            | Text       | Returned status
| Joined from Returns table                  |

---

## **2. Returns Table**

| Column Name | Data Type | Description                  | Transformations /
Notes         |
|-------------|-----------|------------------------------|------------------
---------------|
| OrderID     | Text      | Order identifier             | Used for join to
Orders         |
| Returned    | Text      | Returned status (Yes/No)     | Standardized
|
| Market      | Text      | Market segment               | Standardized
|

---

## **3. People Table**

| Column Name | Data Type | Description                  | Transformations /
Notes         |
|-------------|-----------|------------------------------|------------------
---------------|
| Person      | Text      | Salesperson name             | Used for join to
Orders         |
| Region      | Text      | Sales region                 | Used for RLS,
standardized      |

---

## **4. Date Table**

| Column Name         | Data Type | Description
| Transformations / Notes           |
|---------------------|-----------|----------------------------------------
-----|----------------------------------|
| Date                | Date      | Calendar date
| Generated from Order Date         |
| Date ID             | Integer   | YYYYMMDD format
| Calculated                        |
| Year                | Integer   | Calendar year
| Calculated                        |
| Quarter             | Integer   | Calendar quarter
| Calculated                        |
```

Month	Integer	Calendar month	Calculated
Month Name	Text	Month name	Calculated
Week	Integer	Week number	Calculated
ISO Week	Integer	ISO week number	Calculated
Day of Week	Integer	Day of week (0=Sunday)	Calculated
Day Name	Text	Day name	Calculated
Fiscal Year	Integer	Fiscal year (starts April)	Calculated
Fiscal Quarter	Integer	Fiscal quarter	Calculated
Fiscal Month	Integer	Fiscal month	Calculated
Is Weekend	Boolean	Weekend indicator	Calculated
Is Holiday	Boolean	US holiday indicator	Calculated
Is Business Day	Boolean	Business day indicator	Calculated
Season	Text	Season (Spring, Summer, etc.)	Calculated
Short Date	Text	MM/DD/YYYY format	Calculated
Long Date	Text	Full date format	Calculated
Week Start Date	Date	Start of week	Calculated
Month Start Date	Date	Start of month	Calculated
Quarter Start Date	Date	Start of quarter	Calculated
Days from Today	Integer	Days difference from today	Calculated
Relative Period	Text	Last 30/90 days indicator	Calculated
Previous Year	Date	Date last year	Calculated
Previous Quarter	Date	Date last quarter	Calculated
Previous Month	Date	Date last month	Calculated
YTD, QTD, MTD	Boolean	Year/Quarter/Month-to-date indicators	Calculated
Same Period Last Year	Date	Same period last year	Calculated

```
---

## **5. Measures (DAX)**

| Measure Name                | Formula / Description
|
|-----------------------------|-----------------------------------------
----------------------|
| Total Sales                 | `SUM('Orders'[Sales])`
|
| Total Profit                | `SUM('Orders'[Profit])`
|
| Total Orders                | `DISTINCTCOUNT('Orders'[Order ID])`
|
| Profit Margin %             | `DIVIDE([Total Profit], [Total Sales], 0)`
|
| Average Order Value         | `DIVIDE([Total Sales], [Total Orders], 0)`
|
| YoY Sales Growth %          | `DIVIDE([Total Sales] - [Same Period Last
Year Sales], [Same Period Last Year Sales], 0)` |
| Same Period Last Year Sales | `CALCULATE([Total Sales],
SAMEPERIODLASTYEAR('Date'[Date]))`       |
| YTD Sales                   | `CALCULATE([Total Sales],
DATESYTD('Date'[Date]))`                 |
| Previous YTD Sales          | `CALCULATE([Total Sales],
DATESYTD(SAMEPERIODLASTYEAR('Date'[Date])))` |
| Top 10 Customers by Sales   | DAX using `RANKX` and `TOPN`
|
| Top 10 Products by Profit   | DAX using `RANKX` and `TOPN`
|
| Market Share %              | `DIVIDE([Total Sales], CALCULATE([Total
Sales], ALL('Orders')), 0)`|
| Customer Retention Rate     | DAX using `INTERSECT` of customer sets
|
| Revenue Trend Arrow         | DAX using thresholds for ▲ ▼ ►
|
| Revenue KPI Color           | DAX using thresholds for green/yellow/red
|
| Revenue Variance Indicator  | DAX for Favorable/Unfavorable/Neutral
|

---

## **6. Transformations & Data Quality**

- **Duplicates removed** (OrderID, CustomerID, ProductID)
- **Nulls handled** in critical columns (Sales, Profit, Dates, IDs)
- **Date columns** converted from Excel serials
- **Calculated columns** for delivery days, profit margin, sales category,
```

```
customer tier
- **Joins**: Returns (OrderID), People (SalespersonID)
- **Data validation**: Numeric ranges, text formatting, date logic

---

## **7. Security & Roles**

- **Row-Level Security**: Region, Country, Salesperson, Director, Executive
roles
- **Mapping tables** for multi-region access

---

## **8. Visuals & Formatting**

- **Conditional formatting**: Profit, growth, margin, sales
- **Dynamic titles/labels**: Date range, region, period comparisons
- **KPI cards**: Color-coded, trend arrows, variance indicators
- **Tooltip pages**: Customer, product, region, time-based details

---

**This data dictionary summarizes the structure, columns, measures, and key
transformations in your Power BI model based on the agentic_demo.pbip report
and the global_sales.xlsx dataset.**
```

3. Data Sensitivity Labelling

- Suggests PowerShell scripts to apply sensitivity labels (Confidential, Restricted) automatically to datasets.

```
# PowerShell Script to Apply Data Sensitivity Labels to Power BI Datasets
# Requires Power BI Service Admin permissions and MicrosoftPowerBIMgmt
module


param(
    [string]$WorkspaceId,
    [string]$DatasetId,
    [string]$LabelName,
    [string]$ClientId,
    [string]$ClientSecret,
    [string]$TenantId
)

# Install PowerBI module if not present
```

```
if (-not (Get-Module -ListAvailable -Name MicrosoftPowerBIMgmt)) {
    Install-Module -Name MicrosoftPowerBIMgmt -Force -Scope CurrentUser
}

# Create credential for service principal authentication
$securePassword = ConvertTo-SecureString $ClientSecret -AsPlainText -Force
$credential = New-Object
System.Management.Automation.PSCredential($ClientId, $securePassword)

try {
    # Connect using service principal
    Connect-PowerBIServiceAccount -ServicePrincipal -Credential $credential
-TenantId $TenantId
    Write-Host "Connected to Power BI Service"
}
catch {
    Write-Error "Failed to connect to Power BI: $_"
    exit 1
}

# Verify workspace exists
try {
    $workspace = Get-PowerBIWorkspace -Id $WorkspaceId
    if (-not $workspace) {
        Write-Error "Workspace $WorkspaceId not found"
        exit 1
    }
}
catch {
    Write-Error "Error accessing workspace: $_"
    exit 1
}

# Get and apply sensitivity label
try {
    $labels = Get-PowerBISensitivityLabel
    $selectedLabel = $labels | Where-Object { $_.DisplayName -eq $LabelName
}

    if (-not $selectedLabel) {
        Write-Error "Sensitivity label '$LabelName' not found. Available
labels: $($labels.DisplayName -join ', ')"
        exit 1
    }

    Set-PowerBIDatasetSensitivityLabel -WorkspaceId $WorkspaceId -DatasetId
$DatasetId -LabelId $selectedLabel.Id
    Write-Host "Sensitivity label '$LabelName' applied successfully to
dataset $DatasetId"
}
```

```
catch {
    Write-Error "Failed to apply sensitivity label: $_"
    exit 1
}
```

Prerequisites Setup:

Step 1: Enable Sensitivity Labels in Power BI

1. **Power BI Admin Portal** → **Tenant settings**
2. **Information protection** → **Apply sensitivity labels** → Enable
3. **Microsoft Purview compliance portal** → Create labels if they don't exist

Step 2: Service Principal Permissions

Your service principal needs:

- **Power BI Service Admin** or **Fabric Admin** role
- **Microsoft Graph permissions**: InformationProtectionPolicy.Read

Step 3: File Structure

Save as: `scripts/apply-sensitivity-labels.ps1`

Integration with Your Deployment Pipeline:

Update your GitHub workflow (`.github/workflows/deploy-powerbi.yml`):

```
- name: Apply Sensitivity Labels
  shell: pwsh
  run: |
    ./scripts/apply-sensitivity-labels.ps1 -WorkspaceId
$env:PBI_WORKSPACE_ID -DatasetId "your-dataset-id" -LabelName "Confidential"
-ClientId $env:PBI_CLIENT_ID -ClientSecret $env:PBI_CLIENT_SECRET -TenantId
$env:PBI_TENANT_ID
```

Testing Steps:

Step 1: Check Available Labels

Run this first to see what labels exist:

```
Connect-PowerBIServiceAccount
Get-PowerBISensitivityLabel | Select-Object DisplayName, Id
```

Step 2: Get Dataset ID

```
Get-PowerBIDataset -WorkspaceId "your-workspace-id" | Select-Object Name, Id
```

Step 3: Test the Script

```
./scripts/apply-sensitivity-labels.ps1 -WorkspaceId "workspace-guid" -
DatasetId "dataset-guid" -LabelName "Confidential" -ClientId "client-id" -
ClientSecret "secret" -TenantId "tenant-id"
```